Reinforcement Learning

Introduction

CS3263

Foundations of Artificial Intelligence

Version 5.0

Topics

- Reward Based Learning and Planning
- Prediction (Learning) - Passive RL
  - Adaptive dynamic programming
  - Monte Carlo learning: Direct utility estimation
  - Temporal difference learning
- Control (Planning) - Active RL
  - Active adaptive dynamic programming
  - Exploration vs exploitation
  - Monte Carlo Control
  - Temporal difference Control: Q-learning and SARSA

Recall: Markov Decision Process (MDP)

- Formally:
  - An MDP $M \triangleq (S, A, T, R, \gamma)$ consists of:
  - A set S of states
  - A set A of actions
  - A transition function $T: S \times A \times S \rightarrow [0,1]$ that satisfies the Markov property such that:

$$\forall s \in S, \forall a \in A: \sum_{s' \in S} T(s, a, s') = \sum_{s' \in S} P(s'|s, a) = 1$$

  - A reward function $R: S \rightarrow \Re$ or $R: S \times A \rightarrow \Re$ or $R: S \times A \times S \rightarrow \Re$
  - A discount factor $0 \leq \gamma \leq 1$
  - Solution is a policy – a function to recommend an action in each state: $\pi: S \rightarrow A$
    - Solution involves careful balancing of risk and reward

Recall: Solving MDPs

- What does a solution look like?
  - A policy $\pi: S \rightarrow A$ maps each state s to an action a.
  - The *quality* of a policy is measured by its expected utility over the state sequences it generates.
  - The optimal policy π^* maximizes expected utility. (MEU Principle)
- An MDP agent uses π^*:
  - It observes the current state s
  - Executes the action $a^* = \pi^*(s)$
  - The policy defines how the agent behaves.

Recall: State Utility and Optimal Policy

- Utility of a state s (Value of a state s):
  - Best achievable expected utility starting from s:

$$U(s) = U^{\pi^*}(s) = \max_{\pi} U^{\pi}(s)$$

- Bellman Optimality Equation:

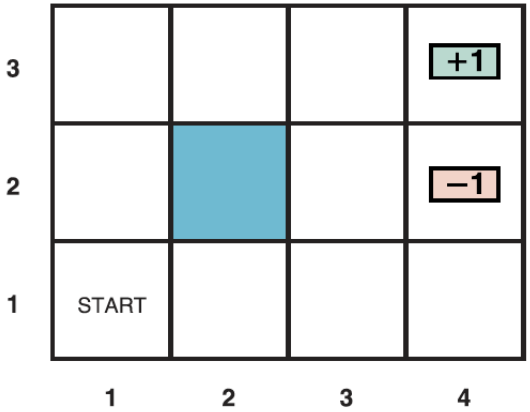
$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Optimal policy (Maximum Expected Utility Principle):

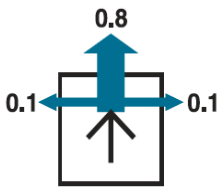
$$\pi^*(s) = \text{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Interpretation:
  - Utility of a state is the maximum expected immediate reward plus discounted utility of successor states.

Example: Navigation in Grid World



(a)

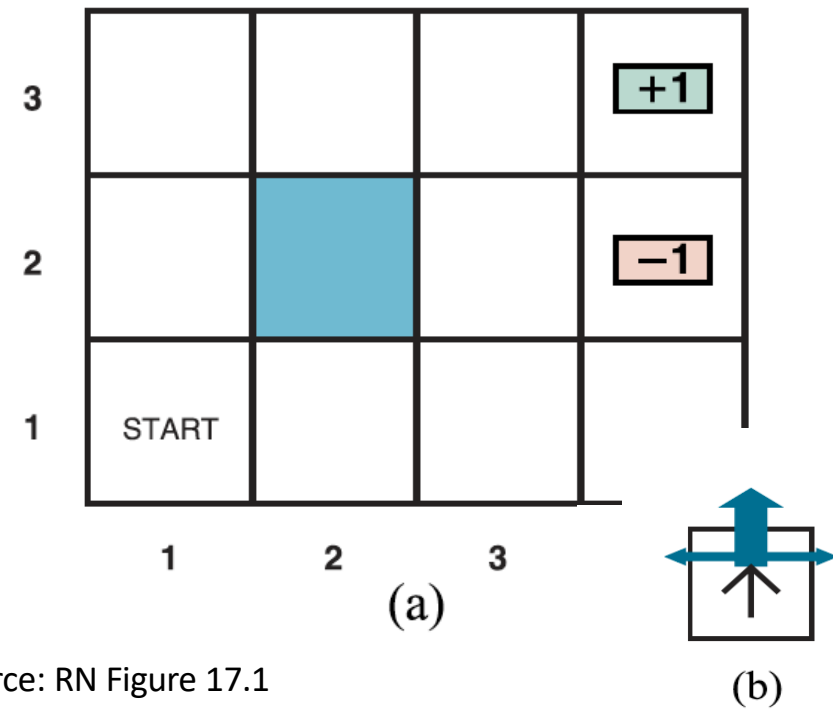


(b)

- Fully observable environment
- States: 4x3,
  - Initial state: (1,1),
  - Terminal states: marked with +1/-1
- Actions:
  - Up, Down, Left, Right

Source: RN Figure 17.1

Example: Navigation in Grid World



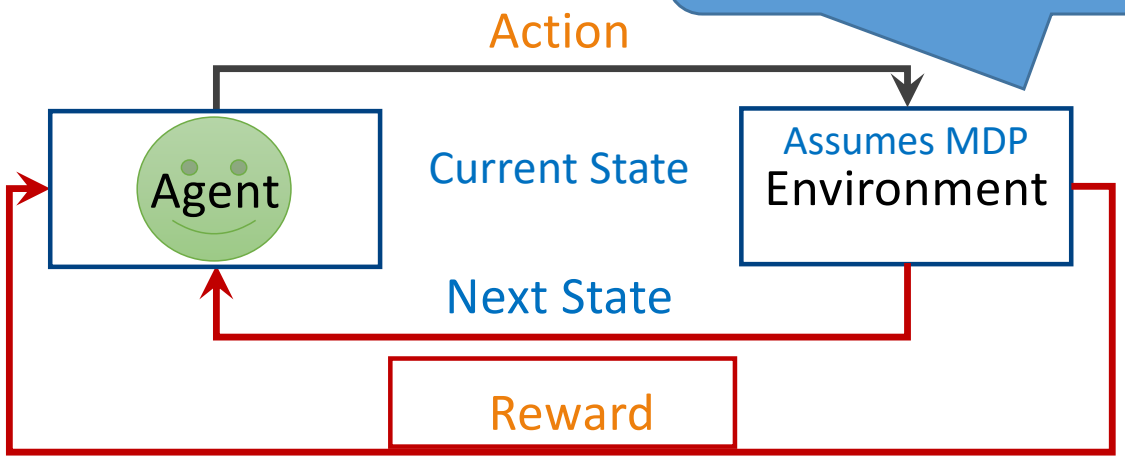
Source: RN Figure 17.1

- Transition model:
  - Unknown
- Reward model:
  - Unknown
- How to find optimal policy π^*?

Reinforcement Learning

- The agent – environment loop

Fully observable environment based on percepts
Unknown dynamics
Unknown action outcomes
Unknown rewards



E.g., playing games, flying helicopter, exploring Mars, assisting elderly

Related work: psychology, neuroscience, operations research, optimal control theory

How to derive optimal policy?

Reward or Reinforcement as feedback for learning – achieved along the way or at the end

Reinforcement Learning Videos

- Stanford Autonomous Helicopter
  - https://youtu.be/M-QUkgk3HyE?si=7yx8E0UeObDJk_lJ
- Run Away from Zombies!
  - https://youtu.be/TWBSSCYnEOM?si=iQdbr6EXEMnJdZYA
- RL in Starcraft II
  - https://youtu.be/gEyBzcPU5-w
- Autonomous robot control
  - https://youtu.be/KyA2uTIQfxw

Types of RL Agents

Assume:
Environment is an MDP
Optimal policy maximizing total rewards

- Reinforcement learning
  - [MB] Model-based: Learns transition model (and reward model) to solve problem
  - [MF] Model- free: Does not (need to) learn transition model to solve problem
- Passive learning agent $\pi \rightarrow U^{\pi}$
  - Learning - Prediction/evaluation/value estimation problem
  - Fixed policy determining behavior
  - Learns utility functions or action-utility functions (Q-functions)
  - sf policy evaluation in policy iteration
- Active learning agent π^{*}
  - Planning - Control problem
  - Learning how to behave – balancing exploration vs exploitation
  - Plans or determines actions to take
  - sf value iteration

Passive Reinforcement Learning $\pi \rightarrow U^{\pi}$

Predict utility function (action-utility function) of state given a fixed policy

Notes: Passive Learning

- Main ideas
  - Agent follows a fixed policy π for multiple trials (episodes)
  - Starts from an initial state s_0 and continues until reaching a terminal state
  - Observes states and rewards while executing π
  - Learns expected utilities under π:

Discounted Rewards received

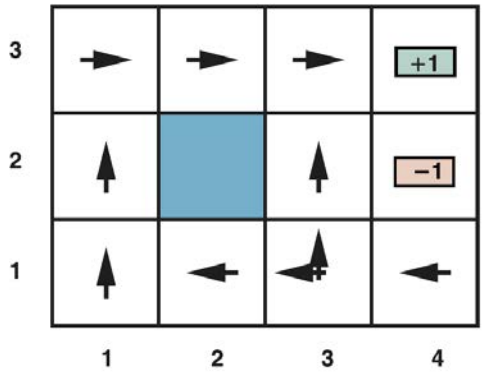
$$U^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) | S_0 = \mathrm{s}\right]$$

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[\sum_{t=0}^{\infty} \gamma^t \, R(S_t, A_t, S_{t+1}) \;\; | \;\; S_0 = s, A_0 = a]$$

  - This is policy evaluation in policy iteration.

Learning Utility Function

$r_{non-terminal} = -0.04$



(a)

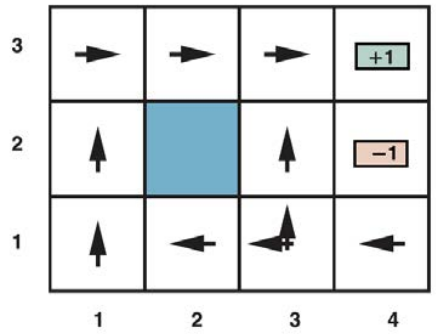
Source: RN Figure 22.1

- The policy π is fixed:
  - Agent always executes $\pi(s)$ in state s
- Goal: Learn utility function or value function $U^{\pi}(s)$ from observations
  - Unknown transition model, $P(s'|s,a)$
  - Unknown reward function, $R(s)$ or $R(s,a,s')$

If known, simply do policy evaluation!

Notes: Trials, Returns, and Expected Returns

$r_{non-terminal} = -0.04$



- Trials
  - Also called state-action sequences/runs/trajectories/episodes
- Returns (G_t)
  - Discounted sum of rewards of a trial from time t onwards

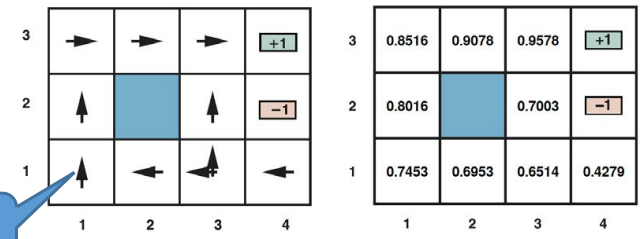
$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots$$

  - Example: $G_t = -0.04 + \gamma(-0.04) + \gamma^2(-0.04) + \cdots + \gamma^n(1)$
- Expected Returns
  - Utility / value of a state (under policy π):

$$U^\pi(s) = E_\pi[\textstyle\sum_{t=0}^{\infty} \gamma^t R(S_t) | S_0 = s] = E_\pi[G_0 | \, S_0 = s]$$

- Recall: Q-function (Action-utility function) $Q(s, a)$
  - Expected total discounted reward if action a is taken in state s, followed by optimal behavior
  - Related to state utilities: $U(s) = \max_a Q(s, a)$

Learning Utility Function



Start state

- Agent executes a series of trials / episodes /runs using π, e.g.,
  - Trial 1 $(1,1)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (2,3)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (4,3)_{\mathbf{+1}}$
  - Trial 2 $(1,1)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (2,3)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (3,2)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (4,3)_{\mathbf{+1}}$
  - Trial 3. $(1,1)_{\mathbf{-.04}} \rightsquigarrow (2,1)_{\mathbf{-.04}} \rightsquigarrow (3,1)_{\mathbf{-.04}} \rightsquigarrow (3,2)_{\mathbf{-.04}} \rightsquigarrow (4,2)_{\mathbf{-1}}$.

Return G_t:
Sum of discounted rewards of a trial

- Utility or value of a state under π:
  - Expected total reward from that state onwards following π
  - Also called expected reward-to-go / expected return

From now on:
Assume $\gamma = 1$

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t) \middle| S_0 = s\right] = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \middle| S_0 = s\right]$$

G_t

Learning Utility Function

- Unknown transition dynamics, unknow reward functions
- **Model-based**
  - Learn the transition and reward functions, then solve the MDP (U^{π})
  - Examples: Adaptive Dynamic Programming (ADP)
- Model-free
  - Directly learn the utility function (U^{π})
  - Examples: Monte Carlo (MC) Learning, Temporal Difference (TD) Learning

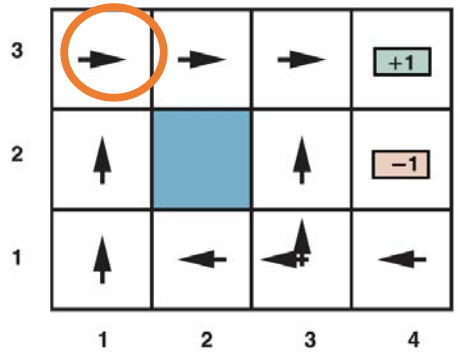
Example: Adaptive Dynamic Programming

$$(1,1)_{\text{-.04}} \rightsquigarrow (1,2)_{\text{-.04}} \rightsquigarrow (1,3)_{\text{-.04}} \rightsquigarrow (1,2)_{\text{-.04}} \rightsquigarrow (1,3)_{\text{-.04}} \rightsquigarrow (2,3)_{\text{-.04}} \rightsquigarrow (3,3)_{\text{-.04}} \rightsquigarrow (4,3)_{\text{+1}}$$
$$(1,1)_{\text{-.04}} \rightsquigarrow (1,2)_{\text{-.04}} \rightsquigarrow (1,3)_{\text{-.04}} \rightsquigarrow (2,3)_{\text{-.04}} \rightsquigarrow (3,3)_{\text{-.04}} \rightsquigarrow (3,2)_{\text{-.04}} \rightsquigarrow (3,3)_{\text{-.04}} \rightsquigarrow (4,3)_{\text{-1}}$$
$$(1,1)_{\text{-.04}} \rightsquigarrow (2,1)_{\text{-.04}} \rightsquigarrow (3,1)_{\text{-.04}} \rightsquigarrow (3,2)_{\text{-.04}} \rightsquigarrow (4,2)_{\text{-1}} \ .$$

What is the transition probability function from $s = (3,3)$?

To compute:

- $P(s' = (3,2) | s = (3,3), a = Right) =$
- $P(s' = (4,3) | s = (3,3), a = Right) =$



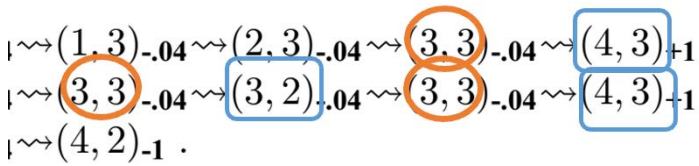
Why are all the actions $Right$?

Adaptive Dynamic Programming (ADP)

- Learn model from experience:
  - Estimate transition probabilities $P(s'|s,a)$ by counting and normalizing observed outcomes.
  - Learn rewards $R(s,a,s')$ when entering state s'.
- Evaluate policy using learned model:

$$U^{\pi}(s) = \sum_{s'} P(s'|s,\pi(s))[R(s,\pi(s),s') + \gamma U^{\pi}(s')]$$

- Notes:
  - Solve $|S|$ linear equations in $O(|S|^3)$ time
  - Learning $P(s'|s,a)$ is supervised learning:
    Input: a state–action pair (s,a), target: observed next state s'.
  - Learning $R(s,a,s')$ is supervised learning:
    Input: a transition (s,a,s'), target: observed reward.
  - ADP enforces global consistency among utilities using Bellman expectation equations.



Notes: ADP-Learner

function PASSIVE-ADP-LEARNER($percept$) **returns** an action
  inputs: $percept$, a percept indicating the current state s' and reward signal r
  persistent: π, a fixed policy
    mdp, an MDP with model P, rewards R, actions A, discount γ
    U, a table of utilities for states, initially empty
    $N_{s'|s,a}$, a table of outcome count vectors indexed by state and action, initially zero
    s, a, the previous state and action, initially null

Using tabular representation

  if s' is new **then** $U[s'] \leftarrow 0$
  if s is not null **then**
    increment $N_{s'|s,a}[s, a][s']$
    $R[s, a, s'] \leftarrow r$
    add a to $A[s]$
    $\mathbf{P}(\cdot \mid s, a) \leftarrow$ NORMALIZE($N_{s'|s,a}[s, a]$)
    $U \leftarrow$ POLICYEVALUATION(π, U, mdp)
    $s, a \leftarrow s', \pi[s']$
    return a

Reward function R

Transition function T

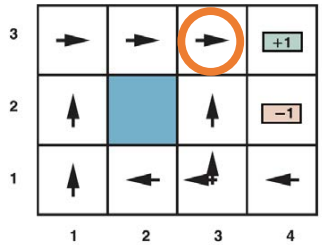
Maximum likelihood estimation (MLE)

Source: RN Figure 22.2

Learning Utility Function

- Unknown transition dynamics, unknow reward functions
- Model-based
  - Learn the transition and reward functions, then solve the MDP (U^{π})
  - Examples: Adaptive Dynamic Programming (ADP)
- **Model-free**
  - Directly learn the utility function(U^{π})
  - Examples: Monte Carlo Learning, Temporal Difference (TD) Learning

Example: Monte Carlo Learning



$$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$$
$$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$$
$$(1,1)_{-.04} \rightsquigarrow (2,1)_{-.04} \rightsquigarrow (3,1)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (4,2)_{-1} \ .$$

- Each trial provides a "sample" of expected return for each state visited
  - In the first trial, sample total reward:
    - For $(1,1) = 0.72$
    - For $(1,2) = 0.76$ and 0.84 and so on
  - Overall:
    - $U^{\pi}(1,1) =$
    - $U^{\pi}(1,2) =$

$$U^{\pi}(s) \leftarrow \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}; \quad G_t^{(i)} = \sum_{k=0}^{\infty} \gamma^k R(S_{t+k})$$

- $N(s)$ = number of times state s has been visited
  - $G_t^{(i)}$ = return from the i –th visit to s

Monte Carlo Learning

- Goal: Estimate state or state–action utilities under a fixed policy.
- Method: Run trials (episodes), then update utilities using the observed return G
- Learning: Update by averaging returns across visits to each state or (state, action).
- Key idea:

  The return G (total discounted reward) is an unbiased estimate of true utility:

  $$\mathbb{E}[G] = U^{\pi}(s) \text{ or } Q^{\pi}(s, a)$$

- Properties:
  - No model required
  - High variance, slow convergence → Does not enforce Bellman consistency during learning
  - Updates only after each episode completes
  - Useful when episodes are short and full returns are observable.

Example: Temporal-Difference (TD) Learning

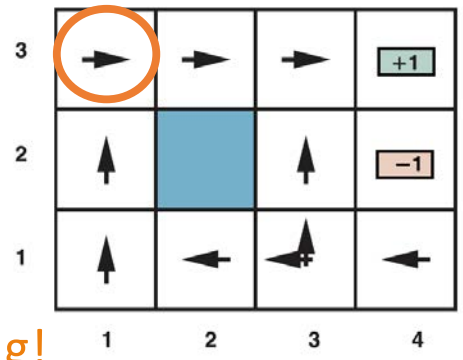
$$(1,1)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (2,3)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (4,3)_{\mathbf{+1}}$$
$$(1,1)_{\mathbf{-.04}} \rightsquigarrow (1,2)_{\mathbf{-.04}} \rightsquigarrow (1,3)_{\mathbf{-.04}} \rightsquigarrow (2,3)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (3,2)_{\mathbf{-.04}} \rightsquigarrow (3,3)_{\mathbf{-.04}} \rightsquigarrow (4,3)_{\mathbf{+1}}$$
$$(1,1)_{\mathbf{-.04}} \rightsquigarrow (2,1)_{\mathbf{-.04}} \rightsquigarrow (3,1)_{\mathbf{-.04}} \rightsquigarrow (3,2)_{\mathbf{-.04}} \rightsquigarrow (4,2)_{\mathbf{-1}} \ .$$

After first trial:

- $U^{\pi}(1,3) = \frac{0.8+0.88}{2} = 0.84$
- $U^{\pi}(2,3) = 0.92$

- In the long run, must obey:
  - $U^{\pi}(1,3) = -0.04 + U^{\pi}(2,3)$
- Hence:
  - $U^{\pi}(1,3) = 0.88$ Current value of 0.84 needs updating!

Solving the MDP without learning a transition function.



Example: Temporal Difference (TD) Learning

- Use Temporal Difference (TD) Update Rule

From now on:
Assume $\gamma = 1$

$$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha[R + \gamma U^{\pi}(s') - U^{\pi}(s)]$$

$$U^{\pi}(1,3) \leftarrow 0.84 + \alpha \cdot [-0.04 + 0.92 - 0.84] = 0.84 + \alpha \cdot 0.04$$

  - If $\alpha = 0.5$, then:

$$U^{\pi}(1,3) = 0.84 + 0.5 \cdot 0.04 = 0.86$$

Closer to the long run value 0.88!

- What is α?
  - α is the learning rate
  - Controls how much each new experience affects the current utility estimate:
    - $\alpha = 1$: full update using the new estimate
    - $\alpha = 0$: ignore the new estimate
  - Typically: $0 < \alpha < 1$
  - For convergence: use decreasing $\boldsymbol{\alpha}$, e.g., $\alpha(n) = \frac{1}{n}$

Temporal Difference Learning (TD)

Update rule uses difference in utlities between successive states

- TD Update Rule (for a transition $s \to s'$):

$$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha\left(\underbrace{R(s,\pi(s),s') + \gamma U^{\pi}(s')}_{\text{TD Target}} - U^{\pi}(s)\right)$$

TD Term or TD Error

  - TD Target: $R + \gamma U^{\pi}(s')$
  - TD Error: $R(s) + \gamma U^{\pi}(s') - U^{\pi}(s)$
  - Learning rate: α, typically $\alpha(n) = O(1/n)$ over visits
- Key Insights
  - Bootstrapping: Updates based on observed transitions and current estimate of $U^{\pi}(s')$
  - Faster learning: TD updates after every step (vs. MC: end of episode)
  - No model required: Unlike Adaptive Dynamic Programming (ADP)
  - Converges: When every state is visited infinitely often, and α decays appropriately

Notes: TD-Learner (Model-Free RL)

function PASSIVE-TD-LEARNER($percept$) **returns** an action
  inputs: $percept$, a percept indicating the current state s' and reward signal r
  persistent: π, a fixed policy
    s, the previous state, initially null
    U, a table of utilities for states, initially empty
    N_s, a table of frequencies for states, initially zero

  if s' is new **then** $U[s'] \leftarrow 0$
  if s is not null **then**
    increment $N_s[s]$
    $U[s] \leftarrow U[s] + \alpha(N_s[s]) \times (r + \gamma\, U[s'] - U[s])$
  $s \leftarrow s'$
  return $\pi[s']$

TD Target TD Term or TD Error

- TD needs only observed transitions to do the updates; no transition model is needed!
- Learn from every experience – update each time we experience s, $\pi(s)$, and s'

Source: RN Figure 22.4

Notes: n-step TD

- n-step return (under policy π):

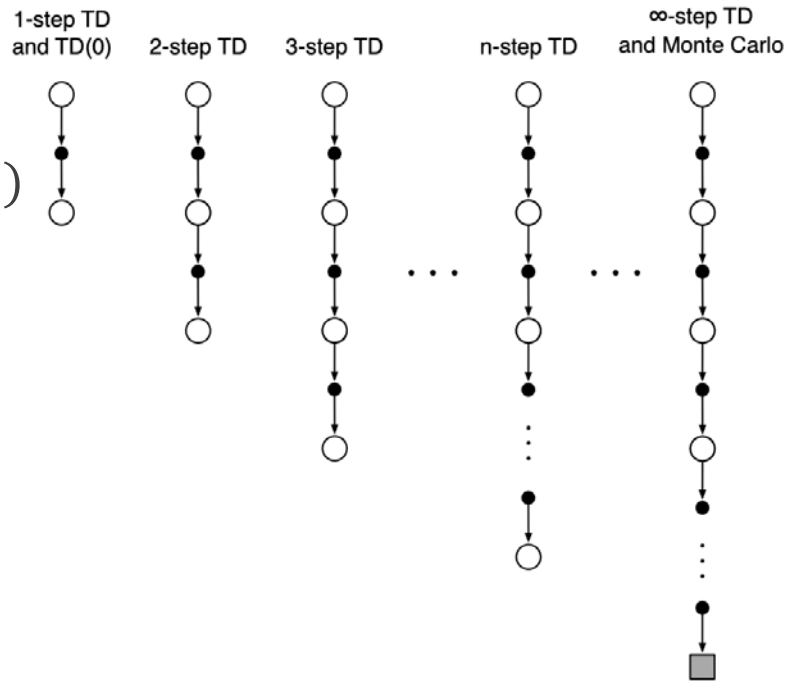
$$G_t^{(n)} = R_t + \gamma R_{t+1} + \cdots + \gamma^{n-1} R_{t+n-1} + \gamma^n U^\pi(S_{t+n})$$

- n-step TD update:

$$U^\pi(S_t) \leftarrow U^\pi(S_t) + \alpha(G_t^{(n)} - U^\pi(S_t))$$

where $U^\pi(S_{t+n})$ is the estimated value at state S_{t+n}

- Note:
  - n-step TD sets $G_t^{(n)}$ as target for update
  - TD sets 1-step return as target for update
  - ∞-step TD is Monte Carlo learning



Source: SB Figure 7.1

Notes: TD(λ)

- Average over different values of n instead of n-step returns
  - Target for the update (λ-update):

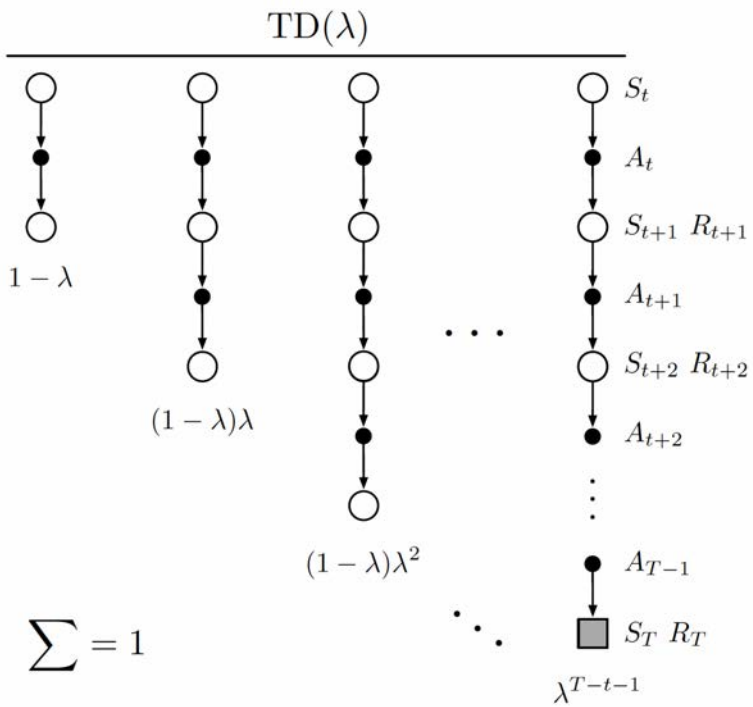
$$u_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} u_{t:t+n}$$

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

  - A weighted sum of n-step returns; nth step is weighted by λ^{n-1}
  - Can be computed efficiently by **eligibility traces** method
- Normalizing constant:

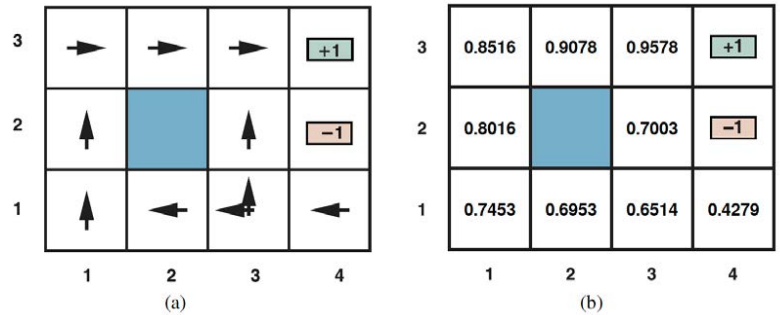
$$\sum_{n=1}^{\infty} \lambda^{n-1} = \sum_{n=0}^{\infty} \lambda^{n} = \frac{1}{1-\lambda}$$

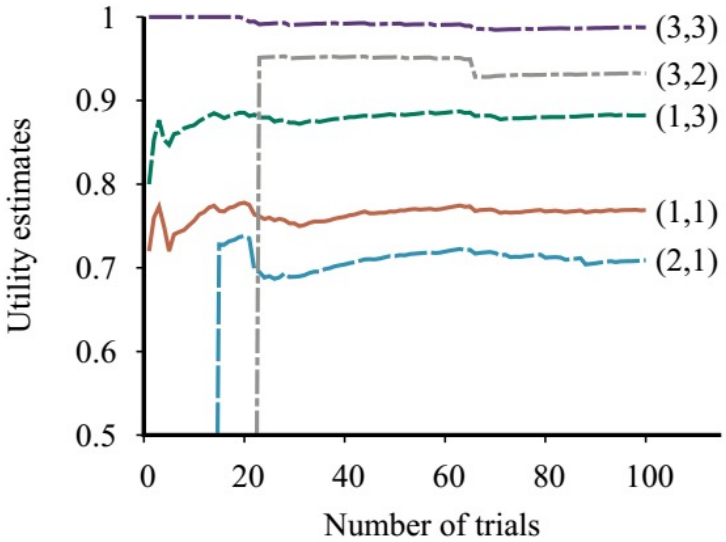
- Converges to TD as $\lambda \to 0$, and to MC as $\lambda \to 1$
  - How to prove this?



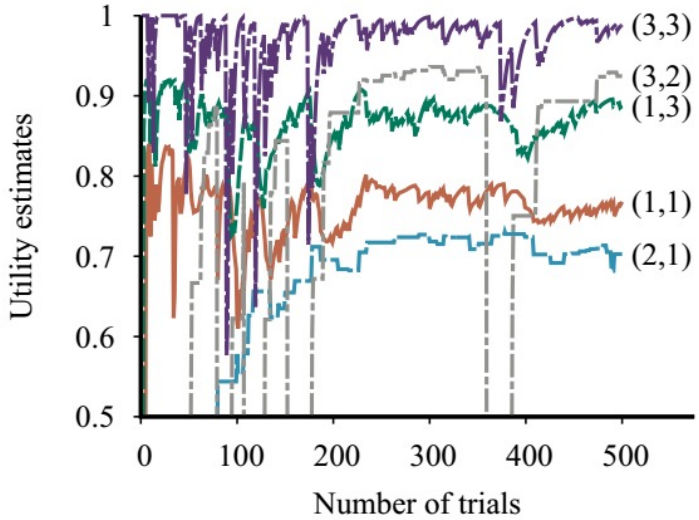
Source: SB Figure 12.1

Example: ADP vs TD





Adaptive Dynamic Programming



Temporal Difference Learning

Source: RN Figure 22.3 (a) and 22.5 (a)

ADP vs TD vs MC

Aspect	ADP	TD(0)	MC
Core equation	$U^\pi(s) = \sum_{s'} P(s'\|s,\pi(s))[R(s,\pi(s),s') + \gamma U^\pi(s')]$	$U^\pi(s) \leftarrow U^\pi(s) + \alpha(R(s,\pi(s),s') + \gamma U^\pi(s') - U^\pi(s))$	$U^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}$ where $G_t^{(i)} = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
Update style	Makes each state agree with **all possible successors** using model T (full Bellman backup)	Makes each state agree with its **observed successor** (bootstraps from estimates)	Makes each state agree with the **full observed return** (no bootstrapping)
Updates per observed transition	**Many/global** (sweeps all states using model)	**One/local** (only visited state)	**Many/local** (updates each visited state after episode ends)
Model required	Needs **known/learned model** (T, R)	✗ **Model-free**	✗ **Model-free**
Data efficiency	**More data-efficient** (can simulate from model)	**Moderate** (needs many samples but updates early)	**Less data-efficient**(waits for full episodes)
Computation per update	**Heavy** (solving Bellman equations or expectations)	**Light** (incremental updates)	**Light** (averaging returns, but delayed)
When updates occur	After model learning or planning sweep	After **each step**	After **each episode**
Bootstrapping?	Yes — uses current estimates of $U^\pi(s')$, with expectations over successors from the model	Yes — uses current estimate of $U^\pi(s')$ for the observed successor	No — uses actual returns G, not current estimates
Summary	**Lower variance,** model-dependent accuracy; bootstraps from model-based expectations	**Lower variance, some bias**; bootstraps from samples	**High variance, no bias**; no bootstrapping

Active Reinforcement Learning $\text{Learn } \pi^*$

Choose to act optimally based on prediction of utility or value of states

Notes: Active Reinforcement Learning

- Main ideas
  - T (transition model) and R (reward function) are unknown
  - Agent can choose any action a at each step
  - Learns an optimal policy π^* that satisfies the Bellman optimality equations:

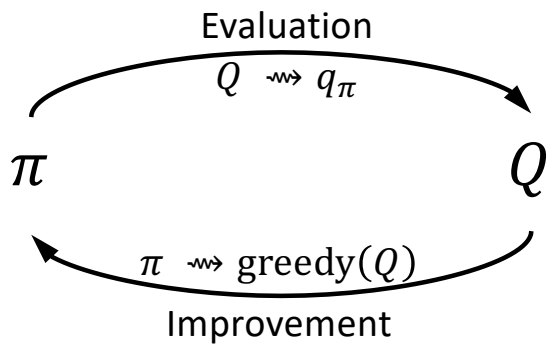
$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

$$Q^*(s,a) = \sum_{s'} P\,(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

  - Must balance exploration vs exploitation.
  - If the learned model $\hat{T}$ is inaccurate, the resulting π^* may be suboptimal (policy loss).

Generalized Policy Iteration

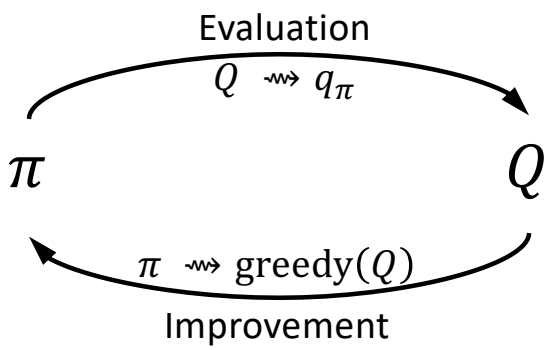
- Generalized policy iteration (GPI):



- Repeatedly alternate between:
  - Policy evaluation — estimate utilities or values of the current policy (e.g., using ADP, Monte Carlo or Temporal Difference methods)
  - Policy improvement — act greedily with respect to the current utility estimates to refine the policy and gather better samples
- Key insight
  - These two processes interleave and influence each other
  - This feedback loop converges to the optimal policy

Source: SB page 86

Notes: GPI and Active RL Techniques



- How it connects
  - Active ADP: does explicit evaluation + greedy improvement
  - SARSA / Q-learning: do incremental TD evaluation; policy becomes greedier as estimates improve
  - GPI helps us see these methods as different ways to realize the same loop

Learning Optimal Policy

- Unknown transition dynamics, unknown reward functions
- **Model-based**
  - Learn the transition and reward functions, then solve the MDP ($\pi*$)
  - Examples: Active Adaptive Dynamic Programming
- Model-free
  - Directly learn the optimal policy $\pi*$
  - Examples: Monte Carlo Control, Q-Learning, SARSA

Active Adaptive Dynamic Programming

- Learn model T and R from passive ADP:
  - Estimate transitions $P(s'|s,a)$
  - Estimate rewards $R(s,a,s')$
- Learn optimal utilities via Bellman optimality equation:

$$U(s) = U^{\pi^*}(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')] = V(s)$$

- Extract optimal policy:
  - $\pi^*(s) = \text{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$

Notes: From Passive to Active ADP-Learner

function ~~PASSIVE~~-ADP-LEARNER(*percept*) **returns** an action
  inputs: *percept*, a percept indicating the current state s' and reward signal r
  persistent: ~~π, a fixed policy~~ ← π not fixed
    mdp, an MDP with model P, rewards R, actions A, discount γ
    U, a table of utilities for states, initially empty
    $N_{s'|s,a}$, a table of outcome count vectors indexed by state and action, initially zero
    s, a, the previous state and action, initially null

Using tabular representation

  if s' is new **then** $U[s'] \leftarrow 0$
  if s is not null **then**
    increment $N_{s'|s,a}[s, a][s']$
    $R[s, a, s'] \leftarrow r$
    add a to $A[s]$
    $\mathbf{P}(\cdot \mid s, a) \leftarrow$ NORMALIZE($N_{s'|s,a}[s, a]$)
    ~~$U \leftarrow$ POLICYEVALUATION(π, U, mdp)~~ ← $U \leftarrow \text{PolicyIteration}(\pi, U, mdp)$
    $s, a \leftarrow s', \pi[s']$
    return a

Count action oucomes

Reward function R

Transition function T

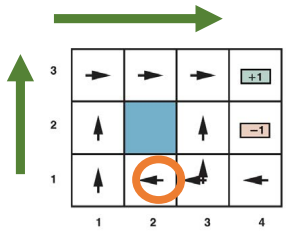
Question:

Is this a good reinforcement learning algorithm?

- π is no longer fixed; changes as transitions & rewards are learned
- Replace POLICY-EVALUATION step by $\pi \leftarrow$ POLICY-ITERATION(mdp) above

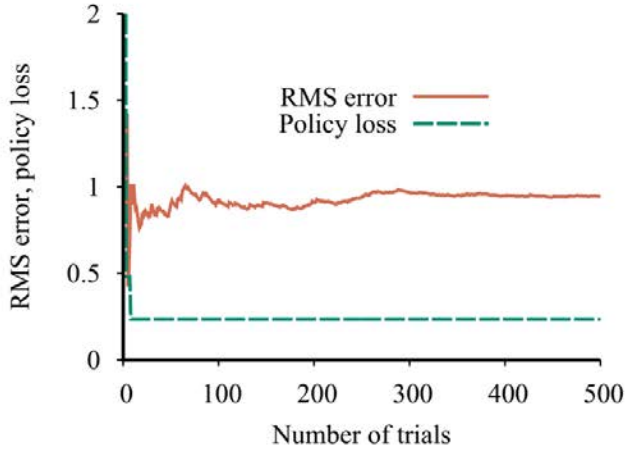
Source: RN Figure 22.2

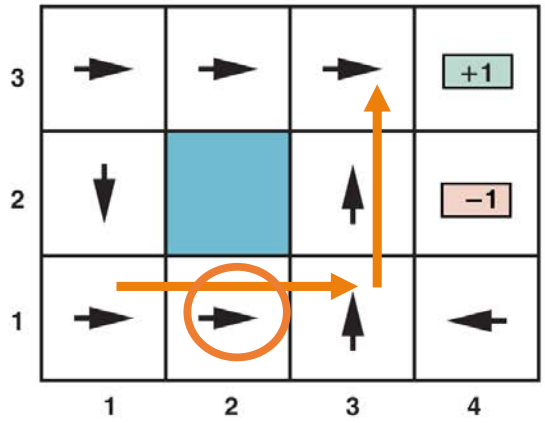
Greedy ADP Learner



RMS error in the utility estimates averaged over non-terminal states

Policy loss: $U^{\pi^*} - U^{\pi}$





Source: RN Figure 22.6

- Learns model T and R through experience (passive ADP)
- Acts greedily: selects actions optimal under the current learned model
- May converge to a suboptimal policy
- Greedy behavior can limit exploration and future learning
- Actions influence both:
  - Immediate reward, and
  - Future data for improving the model

Exploration Vs Exploitation

- Goal: Find an optimal policy that maximizes long-term rewards
- Exploration:
  - Try new actions to learn the true environment T → may lead to better rewards later

Hope to learn something new about the problem!

- Exploitation:
  - Use the current model T to choose actions with the highest estimated rewards now
- Key challenges:
  - How to balance exploring enough to learn vs exploiting to earn?
  - When to stop exploring as knowledge improves?
- As the model becomes accurate, exploration should decrease

Greedy in the Limit of Infinite Exploration

- GLIE: Balancing exploration and exploitation
  - Tries every action in every state infinitely often
  - Prevents missing the optimal action
  - Becomes greedy over time → converges to optimal policy
- GLIE-based Exploration in Reinforcement Learning
  - ϵ-greedy exploration
  - Greedy action with $1 - \epsilon$; random with ϵ
  - Converges slowly
- Improvements
  - Decay ϵ (e.g. $\epsilon = 1/t$)
  - Use *exploration functions* (favor rarely tried actions)

Optimism

- Encourages exploration by being optimistic about untried actions
- Use greedy selection w.r.t. an optimistic utility estimate $U^+(s)$
- Update rule using an exploration function $f(u, n)$:

$$U^+(s) \leftarrow \max_a f\left(\sum_{s'} P(s'|s,a)[R(s,\mathrm{a},\mathrm{s}') + \gamma U^+(s')], N(s,a)\right)$$

  - $N(s,a)$: number of times action a has been tried in state s

- Less-tried actions get inflated estimates → more likely to be explored

Notes: Exploration Functions

- When to explore?
  - Goal: Decide when to explore
  - Random actions explore a fixed amount → inefficient
  - Better: explore actions not yet proven bad
- Exploration function $f(u, n)$:
  - Takes value estimate u and visit count n
  - Returns *optimistic utility* to guide greedy action choice
  - Balances greed (high u) vs curiosity (low n)
  - Should *increase* with u and *decrease* with n

Example: An Exploration Function

- An $f(u, n)$ increasing with u and decreasing with n:

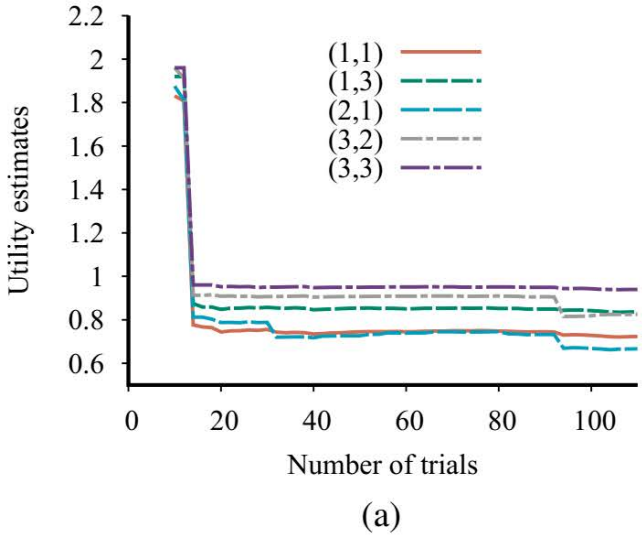
$$f(u,n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{otherwise} \end{cases}$$

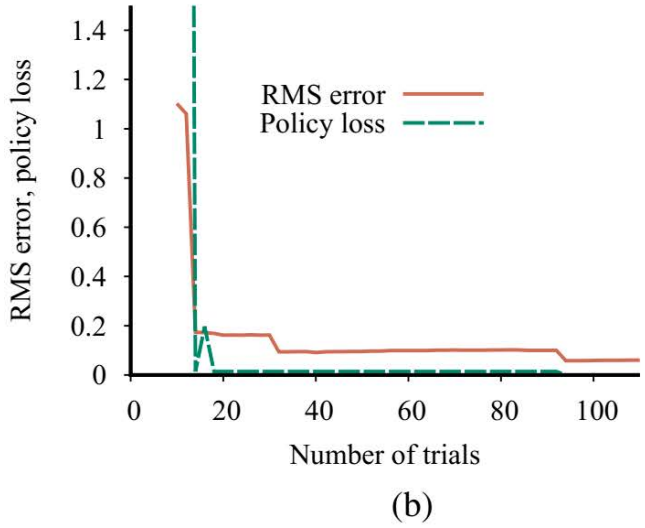
Agent tries each state-action pairs N_e times

Where:

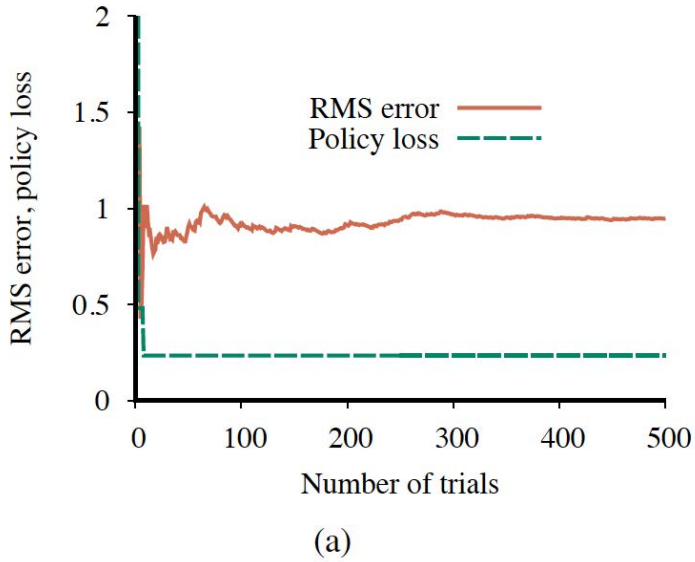
- Inputs: utility or value estimate u and visit count n
- R^+: an optimistic estimate of the best possible reward
- N_e : a fixed parameter

Example: Navigation in Grid World





Exploring ADP



Greedy ADP

- Exploring ADP using exploration function with: $R^+ = 2, N_e = 5$

Source: RN Figure 22.6 (a) 22.7 (a) and (b)

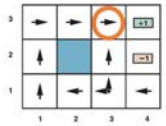
Learning Optimal Policy

- Unknown transition dynamics, unknown reward functions
- Model-based
  - Learn the transition and reward functions, then solve the MDP ($\pi*$)
  - Examples: Active Adaptive Dynamic Programming
- **Model-free**
  - Directly learn the optimal policy $\pi*$
  - Examples: Monte Carlo Control, Q-Learning, SARSA

Monte Carlo Control

$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$
$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$
$(1,1)_{-.04} \rightsquigarrow (2,1)_{-.04} \rightsquigarrow (3,1)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (4,2)_{-1}$.

- Each trial provides a "sample" of expected return for each state visited
  - $U^{\pi}(1,1) = \frac{0.72+0.72-1.24}{3} = 0.067$
  - $U^{\pi}(1,2) = \frac{0.76+0.84+0.76}{3} = 0.79$



- Policy evaluation
  - Estimate action values using **Monte Carlo learning** (no model needed):

$$Q(s,a) \leftarrow \frac{1}{N(s,a)} \sum_{i=1}^{N(s,a)} G_t^{(i)}$$

  - $G_t^{(i)}$: return (discounted cumulative future rewards) from the i-th visit to (s,a)
  - $N(s,a)$: number of times action a has been taken in state s (visits to (s,a))
  - This is the **Monte Carlo estimate** of $Q^{\pi}(s,a)$ (sf. $U^{\pi}(s)$) using episode returns
- Policy improvement
  - Update policy to favor higher-value or underexplored actions:

$$\pi(s) = \arg\max_a f(Q(s,a),\ N(s,a))$$

  - f: **exploration function** encouraging less-visited actions (e.g., optimistic values)

Active TD Control

- Goal:
  - Improve the policy while learning action-utilities $Q(s, a)$ from experience
- Core idea:
  - Use TD updates to estimate action-utilities (no full episodes needed)
  - Use these updated utilities to make the policy greedier over time
- Two main algorithms:
  - SARSA (on-policy): learns from the actions actually taken
  - Q-learning (off-policy): learns from the best possible actions

SARSA

- State–Action–Reward–State–Action

  Model free, on-policy method:

  - Uses Temporal Difference (TD) for policy evaluation
  - Uses ϵ-greedy for action selection

- TD update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\boxed{R(s,a,s') + \gamma Q(s',a') - Q(s,a)}]$$

TD Term or Error

  - a' : the next action actually taken in state s' (from the current policy π)
  - This rule is applied after observing the full quintuple (s, a, r, s', a')

- Convergence:

  - With decreasing ϵ and decreasing learning rates α , and if all state–action pairs are visited infinitely often, SARSA converges to the optimal policy.

Q-Learning: Learning Action-Utility Functions

- Q-learning
  - Model-free: learns $Q(s,a)$ directly from experience; no transition model or lookahead needed
  - Off-policy: Based on the Bellman optimality equation for Q:

$$Q(s,a) = \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma \max_{a'} Q(s',a')\,]$$

  - Learns via Temporal Difference (TD) updates:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\boxed{R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)}]$$

TD Term or Error

- Convergence:
  - With decreasing ϵ and decreasing learning rates α , and if all state–action pairs are visited infinitely often, Q-Learning converges to the optimal policy.

Q-learning: Active TD Learning

function Q-LEARNING-AGENT($percept$) **returns** an action
  inputs: $percept$, a percept indicating the current state s' and reward signal r
  persistent: Q, a table of action values indexed by state and action, initially zero
    N_{sa}, a table of frequencies for state–action pairs, initially zero
    s, a, the previous state and action, initially null

TD Update

  if s is not null **then**
    increment $N_{sa}[s, a]$
    $Q[s, a] \leftarrow Q[s, a] + \alpha(N_{sa}[s, a])(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$
  $s, a \leftarrow s', \text{argmax}_{a'} f(Q[s', a'], N_{sa}[s', a'])$
  return a

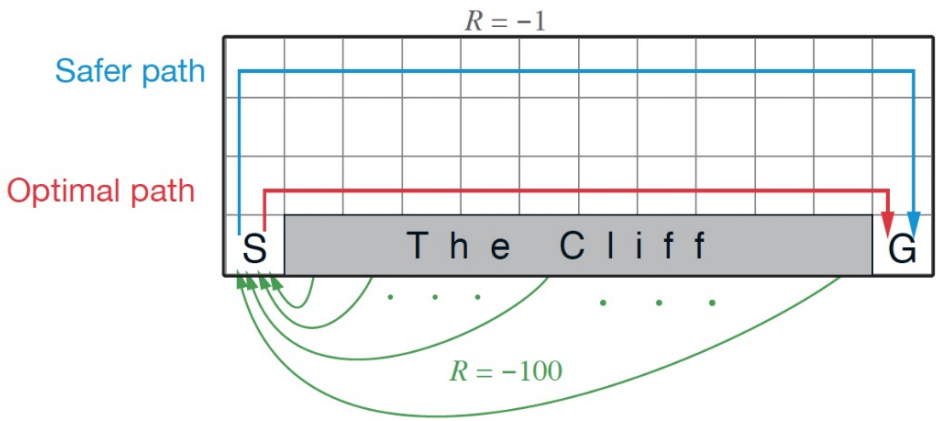
Exploration function

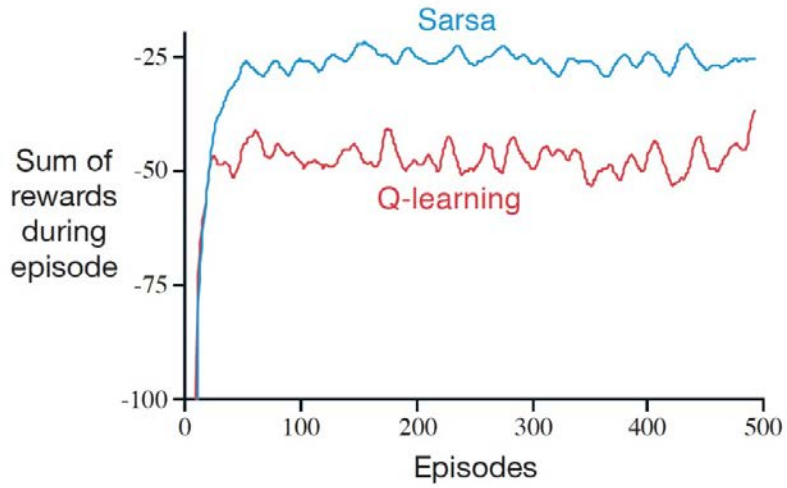
TD Term or Error

- TD Q-learning does not need a transition model, $P(s'|s, a)$ for learning or action selection

Source: RN Figure 22.8

Example: Cliff Walking





- Actions: left, right, up, down
- Reward: -1 for each step; -100 if you fall off
- SARSA & Q-learning used with ϵ-greedy; $\epsilon = 0.1$
  - Why does SARSA outperform Q-learning?
  - What happens if GLIE exploration is used?

Source: SB: Example 6.6

SARSA vs Q-Learning

	SARSA	Q-learning
Type	On-policy — learns from actions actually taken by the current (exploring) policy	Off-policy — learns as if always choosing the best action
Update	$Q(s,a) \leftarrow Q(s,a) + \alpha\big(R(s,a,s') + \gamma Q(s',a') - Q(s,a)\big)$	$Q(s,a) \leftarrow Q(s,a) + \alpha\left(R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$
Target	Uses $Q(s',a')$ from the **next action actually taken**	Uses $\max_{a'} Q(s',a')$ from the **best possible action**
Exploration effect	Penalizes bad exploratory actions → safer, more cautious	Does not bootstrap through the next exploratory action → more optimistic
Learning behavior	Incorporates negative outcomes → can be **slower but more stable**	Ignores negative exploratory outcomes → **faster but riskier**
Similarity	Behaves the same when the agent **never explores** (always takes the best action)	Behaves the same when the agent **never explores** (always takes the best action)
Convergence	Converges if ϵ and α decay and all (s,a) are visited infinitely often	Converges if ϵ and α decay and all (s,a) are visited infinitely often

Summary: Reinforcement Learning

- Passive Learning — fixed policy (policy evaluation) $\pi \rightarrow U^{\pi}$
  - [MB] Adaptive dynamic programming (ADP)
  - [MF] Monte Carlo prediction (MC): direct value (utility) estimation
  - [MF] Temporal-difference (TD) prediction
- Active Learning — optimal policy (control) $\text{Learn } \pi^{*}$
  - [MB] Active ADP (model-based control)
  - [MF] Monte Carlo control
  - [MF] Active TD methods (TD control): Q-learning, SARSA

Homework

- Readings:
  - RN: 22.1, 22.2. 22.3.1, 22.3.2
  - *SB: 6.2, 6.4, 6.5* (Optional)
- Some classical papers and recent surveys on RL

  (Journal articles publicly available online or through NUS Library e-Resources)
  - L. P. Kaelbling, M. L. Littman, & A. W. Moore. Reinforcement Learning: A Survey. JAIR, 4:237-285, 1996
  - Wirth, C., et al., *A survey of preference-based reinforcement learning methods*. J. Mach. Learn. Res., 2017. **18**(1): p. 4945–4990.